

Resolution: Proof Example

Video Lesson

Duration: 00:11:25

Slide Deck

M09S08 (<https://canvas.tamu.edu/courses/430127/files/92570680?wrap=1>) 

Lesson Transcript

In this lecture, we will finally look at a full-resolution-based proof in first-order logic.

Example Proof Using Resolution ¹



Given: (1) The customs officials searched everyone who entered the country who was not a VIP. (2) Some of the drug dealers entered the country, and they were only searched by drug dealers. (3) No drug dealer was a VIP.

Prove: (4) Some of the customs officials were drug dealers.

Let us consider this example. The customs officials searched everyone who entered the country who was not a VIP, and some of the drug dealers entered the country, and they were only searched by drug dealers. And finally, no drug dealer was a VIP. So, given these three pieces of knowledge, can we prove that some of the customs officials were drug dealers?

Example: Predicates

- ❶ $C(x)$: x is a customs official
- ❷ $E(x)$: x entered the country
- ❸ $V(x)$: x is a VIP
- ❹ $S(x, y)$: x was searched by y
- ❺ $D(x)$: x is a drug dealer



To do a resolution proof, first, we must turn those natural language sentences into first-order logic. The very first step, which is the most important, is to pick the correct predicates to describe the situation.

For this theorem, we use $C(x)$ to represent x as a customs official. $E(x)$ to represent x entered the country. $V(x)$ to represent x is a VIP. $S(x, y)$ to represent x was searched by y . So, here again, the ordering is very important. And finally, $D(x)$ represents x is a drug dealer.

Example: English to First Order Logic

(1) The customs officials searched everyone who entered the country who was not a VIP. (2) Some of the drug dealers entered the country, and they were only searched by drug dealers. (3) No drug dealer was a VIP. (4) Some of the customs officials were drug dealers.

- ① $\forall x((E(x) \wedge \neg V(x)) \rightarrow \exists y(S(x, y) \wedge C(y)))$
- ② $\exists x(E(x) \wedge D(x) \wedge \forall y(S(x, y) \rightarrow D(y)))$
- ③ $\forall x(D(x) \rightarrow \neg V(x))$
- ④ $\exists x(D(x) \wedge C(x))$

In the theorem, we have three sentences in the premises and one sentence as the conclusion. From those, we can get these four first-order logic formulas. So, the first sentence, the customs official searched everyone who entered the country who was not a VIP, is turned into for all x $E(x)$ and not $V(x)$ implies there exists y such that y searched x and y is customs official.

The second sentence: some of the drug dealers entered the country, and they were only searched by drug dealers. That becomes there exists x , such that $E(x)$ and $D(x)$ and for all y who searched x , y is a drug dealer, D .

So, the last sentence in the premises is: no drug dealer was a VIP. Then, for all x drug dealers x , then not VIP x .

Finally, the conclusion is: some of the customs officials were drug dealers. That translates easily to there exists x , such that x is a drug dealer, D , and x is a customs official.

Of course, these are first-order logic formulas, but they are not in the standard form. So, we have to turn each and every one of these four formulas into the standard form.

Example: Standard Form (I)

$$\begin{aligned}
 (1) \quad & \forall x((E(x) \wedge \neg V(x)) \rightarrow \exists y(S(x, y) \wedge C(y))) \\
 \{rm \rightarrow\} &= \forall x(\neg(E(x) \wedge \neg V(x)) \vee \exists y(S(x, y) \wedge C(y))) \\
 \{prenex\} &= \forall x \exists y(\neg E(x) \vee V(x) \vee (S(x, y) \wedge C(y))) \\
 \{skol\} &= \forall x(\neg E(x) \vee V(x) \vee (S(x, f(x)) \wedge C(f(x)))) \\
 \{add()\} &= \forall x((\neg E(x) \vee V(x)) \vee (S(x, f(x)) \wedge C(f(x)))) \\
 \{dist\} &= \forall x(\underbrace{(\neg E(x) \vee V(x) \vee S(x, f(x)))}_{\wedge (\neg E(x) \vee V(x) \vee C(f(x)))})
 \end{aligned}$$

Clauses:

$$(1a) \neg E(x) \vee V(x) \vee S(x, f(x))$$

$$(1b) \neg E(x) \vee V(x) \vee C(f(x))$$

So, here is the first sentence: for all $E(x)$ and not $V(x)$ implies there exists y $S(x, y)$ and $C(y)$. So, we have to go through these steps. First, we eliminate the implication, and then we find the prenex normal form of the second line.

And this is what we get. For all x , there exists y , not $E(x)$ or $V(x)$ or $S(x, y)$ and $C(y)$.

Next, we perform skolemization to get rid of the existential quantifier y . And since there is a preceding universally quantified variable x , we have to replace y with a skolem function $f(x)$. So y here, and y here now become $f(x)$.

And to turn this into conjunctive normal form, we have not $E(x)$ or $V(x)$ or something here in the parentheses. To be able to distribute this whole thing over this disjunction and conjunction, we simply put parentheses over here using the associative property. And once we have done that, we get two clauses: $\neg E(x) \vee V(x) \vee S(x, f(x))$. That is the first clause, Clause 1a. Then we get the second clause, $\neg E(x) \vee V(x) \vee C(f(x))$. So, that is the second clause, Clause 1b. So, we get two clauses from the first sentence.

Example: Standard Form (II)

$$\begin{aligned}
 (2) \quad & \exists x(E(x) \wedge D(x) \wedge \forall y(S(x, y) \rightarrow D(y))) \\
 \{rm \rightarrow\} &= \exists x(E(x) \wedge D(x) \wedge \forall y(\neg S(x, y) \vee D(y))) \\
 \{prenex\} &= \exists x \forall y(E(x) \wedge D(x) \wedge (\neg S(x, y) \vee D(y))) \\
 \{skol\} &= \forall y(\underbrace{E(a)}_{\wedge} \underbrace{D(a)}_{\wedge} \underbrace{(\neg S(a, y) \vee D(y))}_{\wedge})
 \end{aligned}$$

Clauses:

$$(2a) E(a)$$

$$(2b) D(a)$$

$$(2c) \neg S(a, y) \vee D(y)$$

The second sentence is: there exists x such that $E(x)$ and $D(x)$ and for all y $S(x, y)$, then $D(y)$. Again, we remove the implication to go from S implies D to $\neg S(x, y) \vee D(y)$.

Then, we find the prenex normal form by moving this universally quantified variable y and the universal quantifier to the front. And finally, we have these three elements that are connected with conjunction.

Since a single predicate is a clause for these two cases, and here we also have a clause because we have literals connected with this conjunction, so this whole thing results in three clauses after skolemization. Since this existential quantifier appears at the very beginning, we can replace x with a skolem constant, a , and as a result, we get these three clauses from the second sentence.

Example: Standard Form (III)

$$\begin{array}{ll}
 (3) & \forall x(D(x) \rightarrow \neg V(x)) \\
 \{rm \rightarrow\} & = \forall x(\neg D(x) \vee \neg V(x)) \quad \blacktriangleleft \\
 \text{Clause:} & \\
 (3) & \neg D(x) \vee \neg V(x) \\
 (4) & \exists x(D(x) \wedge C(x)) \\
 \{negate\} & \Rightarrow \neg(\exists x(D(x) \wedge C(x))) \\
 \{prenex\} & = \forall x\neg(D(x) \wedge C(x)) \\
 \{CNF\} & = \forall x(\neg D(x) \vee \neg C(x)) \\
 \text{Clause:} & \\
 (4) & \neg D(x) \vee \neg C(x)
 \end{array}$$

This is the third sentence from the premises, the last one, which is: for all x , $D(x)$ implies not $V(x)$. Here, we only need to remove this implication to get our clause, and we are already done. It is in prenex normal form and does not need any skolemization. So, we have this clause: $\neg D(x) \vee \neg V(x)$.

The final sentence is the conclusion sentence. There exists x such that $D(x)$ and $C(x)$. To turn this into a resolution problem, we have to negate it. So, once we negate it, it becomes: not there exists $D(x)$ and $C(x)$. And using this trick of moving this negation inside of the quantifier and flipping it, we get: for all x , not $D(x)$ and $C(x)$.

And using De Morgan's law, we get: not $D(x)$ or not $C(x)$, and this now is a single clause. From this, once we negate and find the normal form, then we get: $\neg D(x) \vee \neg C(x)$.

Example: Clauses

(1a) $\neg E(x) \vee V(x) \vee S(x, f(x))$

(1b) $\neg E(x) \vee V(x) \vee C(f(x))$

(2a) $E(a)$

(2b) $D(a)$

(2c) $\neg S(a, y) \vee D(y)$

(3) $\neg D(x) \vee \neg V(x)$

(4) $\neg D(x) \vee \neg C(x)$

Note: The input to your theorem prover will be in a standard form like the above.

Exercise 1: rewrite the above in LISP representation.

Exercise 2: use resolution to derive **False**.



Collecting all of these resulting clauses from the normal form, we get two clauses from the first sentence, three clauses from the second sentence, one clause from the third sentence, and one clause from the negated conclusion.

This will be the input to a theorem-proving algorithm, and eventually, we want some of these to be resolved and added to the collection. And after several steps, we want to derive **False**.

Example: Clauses

(1a) $\neg E(x) \vee V(x) \vee S(x, f(x))$

(1b) $\neg E(x) \vee V(x) \vee C(f(x))$

(2a) $E(a)$

(2b) $D(a)$

(2c) $\neg S(a, y) \vee D(y)$

(3) $\neg D(x) \vee \neg V(x)$

(4) $\neg D(x) \vee \neg C(x)$

Note: The input to your theorem prover will be in a standard form like the above.

Exercise 1: rewrite the above in LISP representation.

Exercise 2: use resolution to derive **False**.

(5) $V(a) \vee S(a, f(a))$

Let us see how this resolution process goes. First, we identify Clause 1a and Clause 2a to be resolvable, where we can replace x with a and get rid of these two. That will unify these two expressions. As a result, we get $V(a)$ or $S(a, f(a))$, which is that and we put it back into the list of clauses.

Next, we resolve 1b and 2a. Then likewise eliminating $E(a)$ and not $E(x)$. After unification, we get $V(a)$ or $C(f(a))$. And again, we put that clause back into the list.

Next, we resolve Clause 2b and 3 to get rid of $D(a)$ and $\neg D(x)$ by replacing x with a . And as a result, we get $\neg V(a)$. And again, we put that clause into the list of clauses.

Next, we can resolve these two new clauses to get rid of $V(a)$ and $\neg V(a)$ to get $S(a, f(a))$ and put that new clause into the list of clauses. Next, we resolve Clause 2c and Clause 8, where the complementary literal is $\neg S$ and S . So, we get rid of those by replacing y with $f(a)$, and we retrieve $D(f(a))$ as a result and put that in the clause list.

Example: Clauses

- (1a) $\neg E(x) \vee V(x) \vee S(x, f(x))$
- (1b) $\neg E(x) \vee V(x) \vee C(f(x))$
- (2a) $E(a)$
- (2b) $D(a)$
- (2c) $\neg S(a, y) \vee D(y)$
- (3) $\neg D(x) \vee \neg V(x)$
- (4) $\neg D(x) \vee \neg C(x)$

Note: The input to your theorem prover will be in a standard form like the above.

Exercise 1: rewrite the above in LISP representation.

Exercise 2: use resolution to derive **False**.

- (5) $V(a) \vee S(a, f(a))$
 - (6) $V(a) \vee C(f(a))$
 - (7) $\neg V(a)$
 - (8) $S(a, f(a))$
 - (9) $D(f(a))$
 - (10) $C(f(a))$
-

Next, we resolve Clause 6 and Clause 7 to get rid of V and $\neg V$ and retrieve $C(f(a))$. And that goes into the list of clauses as Clause number 10.

Next, we resolve Clause 4 and Clause 10, where we find that there is $\neg C$ and C , and by replacing x with $f(a)$, we can retrieve $\neg D(f(a))$. And we put that into the list of clauses as Clause number 11. Finally, when we resolve Clause 11 and Clause 9, in this case, they can be directly resolved without any unification, then we get **False**, and we put that in the clause list. And finally, we are done.

Example: Clauses

(1a) $\neg E(x) \vee V(x) \vee S(x, f(x))$

(1b) $\neg E(x) \vee V(x) \vee C(f(x))$

(2a) $E(a)$

(2b) $D(a)$

(2c) $\neg S(a, y) \vee D(y)$

(3) $\neg D(x) \vee \neg V(x)$

(4) $\neg D(x) \vee \neg C(x)$

Note: The input to your theorem prover will be in a standard form like the above.

Exercise 1: rewrite the above in LISP representation.

Exercise 2: use resolution to derive **False** .

That concludes this example, resolution proof. In the following lecture, we will look at how to write a full algorithm because, in this example, we just use some common sense rules and ideas to identify which clause could be resolved with which other and proceed based on our intuition. But for fully automated theorem proving, we need an algorithm, and that is the subject of the next lecture.